

Tag Validator Pro

Bulk website tag-validation tool with conversational AI assistant

Version	v3.1 (Hybrid Mode)
Document	Architecture & Stack Reference
Repository	github.com/Ashu09-design/tag-validator-v2
Hosting	Hugging Face Spaces (Docker)
Date	May 2026

1. What it does

Tag Validator Pro is a bulk web-analytics auditing tool. You give it a list of URLs (or a whole domain) and it launches a real headless Chromium browser at each page, intercepts the network traffic via Chrome DevTools Protocol (CDP), and reports exactly which analytics and marketing tags actually fired on the live page.

It answers questions like:

- Is **GA4 / GTM / Adobe Analytics / Tealium** live on these pages?
- What are the concrete IDs (G-XXXX, GTM-XXXX, Adobe report suite, Tealium account/profile/env)?
- Did the **page_view** tag actually fire?
- Which **marketing pixels** (Meta, Google Ads, LinkedIn, TikTok, etc.) fire under each OneTrust consent scenario? Are any firing when they should not (consent violation)?
- Give me the Excel report and email it on a schedule.

2. Tech stack

Layer	Technology	Why
Backend server	Node.js + Express	Lightweight HTTP server, static files, routes.
Browser automation	Python + Playwright + playwright-realcdp	Real Chromium with CDP for accurate network capture.
Network capture	Chrome DevTools Protocol (CDP)	Catches all requests including POST beacons / sendBeacon.
Process bridge	child_process.spawn	Node spawns the Python audit, streams logs to the UI.
Excel I/O (server)	xlsx (SheetJS)	Reads results to build email summaries, serves downloads.
Excel I/O (Python)	pandas + openpyxl	Reads input lists, writes the audit report.
Scheduling	node-cron	Hourly / daily / weekly / monthly automated runs.
Email alerts	Brevo HTTP API	Sends report via HTTPS:443 (works where SMTP is blocked).
AI chatbot	Groq (gpt-oss-120b) + Google Gemini	Tool-falling LLM that drives the validators conversationally.
Frontend	Plain HTML + CSS + vanilla JS	Small UI; no framework, no build step, fast.
Unique IDs	uuid	Stable IDs for scheduled jobs.
Packaging	Docker	Single image with Node, Python, Chromium and all deps.
Hosting	Hugging Face Spaces (Docker) + GitHub	Free Docker hosting + version control.

3. How the pieces fit together

There are three loosely-coupled layers:

- **The Node server** (`server.js`) serves the UI, exposes a REST API, spawns Python subprocesses, streams their logs, parses the resulting Excel, and handles cron + email.

- **The Python audit engine** (`bulk_tag_validator.py`, `domain_crawler.py`) launches Playwright Chromium, hooks CDP, drives the page through cookie banners, captures every request, and writes the Excel report.
- **The browser UI** (`public/index.html` + `public/app.js`) is a glassmorphism single-page UI with three main tabs and a floating AI chatbot.

Communication flow

Browser → Node API → spawn(Python) → Playwright Chromium → target site → CDP capture → Excel/JSON → Node reads result → UI polls /status and renders.

4. Core features

Live Validation tab

Upload an Excel list of URLs and audit them in one of three modes:

- **Tealium / Adobe:** Tealium loaded?, account/profile/env; Adobe loaded?, report suite, page-view tag.
- **GA4 / GTM:** GTM container loaded?, GTM ID; GA4 fired?, Measurement ID, page_view event.
- **Marketing Pixels:** every marketing pixel that fired, under 5 OneTrust consent scenarios (Accept All / Reject All / Performance / Functional / Targeting), with fire counts, pixel IDs, and the source that fired it (Tealium / Adobe / GTM / Hardcoded) attributed from the initiator stack. Compliance FAILs if any marketing pixel fires under Reject All.

Domain Crawl tab

Give a single domain URL and the crawler discovers every reachable same-domain page via sitemap.xml + BFS over hyperlinks. URL normalization makes http/https and www/non-www collapse to one canonical URL so duplicates never appear. Optional path-prefix scope (e.g. only /de/ locale). Set *max pages* = 0 for unlimited. "Crawl + Validate" chains the crawl into a validation in the chosen mode.

Automated Schedules tab

Upload an Excel of URLs, set a frequency (hourly/daily/weekly/monthly), optionally an alert email, and a node-cron job runs the audit on schedule. After every run the summary + failed-site list is emailed via the Brevo HTTP API with the full Excel attached. Audit history is browsable per scheduled job.

Cancel + light/dark theme

Long validations and crawls can be cancelled mid-run (Cancel button). A persistent light/dark theme toggle lives in the navbar.

5. AI Assistant (chatbot)

A floating chat widget ("Tagly") lets users talk to the validator in natural language. The user types a question or pastes a URL; the LLM decides which tool to call, executes it server-side (the same Python validators), and answers in plain language.

LLM providers

- **Primary: Groq** — openai/gpt-oss-120b via Groq's OpenAI-compatible endpoint. Chosen for fast, reliable structured tool calls.
- **Fallback: Google Gemini** — gemini-2.0-flash via Google's OpenAI-compatible endpoint. When Groq is rate-limited or unavailable the assistant auto-switches to Gemini, so the chatbot keeps working past Groq's daily quota.

Tools the LLM can call

Tool	What it does
<code>validate_tags(urls, mode)</code>	Live browser audit. mode <code>full</code> (default) detects everything in one pass
<code>crawl_domain(url, max_pages)</code>	Discovers every same-domain page URL and returns a downloadable Excel.

Things the assistant can answer

- "Is GA4 / GTM / Adobe / Tealium implemented on this URL?" — auto-detects whichever stack is actually there.
- "What is the Measurement ID / GTM ID / report suite / Tealium account?"
- "Does the page_view tag fire?" — answers the page-view status only.
- "Which marketing pixels fire on this site under Accept All?" — reports just the asked scenario from the 5 consent scenarios.
- "What is the CMS / platform of this site?" — detects WordPress, Shopify, Wix, Squarespace, Drupal, Adobe Experience Manager, Webflow, HubSpot CMS, Magento, Sitecore and more.
- "What HTTP status does the page return?" — flags 404 / 500 errors.
- "Crawl this domain and give me all page URLs in Excel."
- "How do I fix GA4 page_view not firing? Give an example." — expert troubleshooting answers from its own knowledge.

Chat UX

- Floating message-bubble button (bottom-right) with a pulsing ring and an animated greeting speech bubble.
- Markdown rendering for replies (tables, code, links).
- **Cancel button:** while the bot is replying the send button turns into a red stop button. Clicking it aborts the request client-side AND the server-side LLM call, freeing the assistant immediately.
- Settings panel for the Groq + Gemini API keys (stored server-side only).

6. API endpoints

Method	Path	Purpose
POST	/api/tag-validator/upload	Upload input Excel for the Live Validation tab.
POST	/api/tag-validator/run	Start a validation run in the chosen mode.
POST	/api/tag-validator/cancel	Cancel the running validation / crawl.
GET	/api/tag-validator/status	Poll progress logs + running flag.
GET	/api/tag-validator/results	JSON of the last validation results.
GET	/api/tag-validator/results-rich	Per-scenario pixel JSON (Pixels mode).
GET	/api/tag-validator/download	Download the last Excel report.
POST	/api/tag-validator/crawl	Crawl a domain, write crawled_urls.xlsx.
POST	/api/tag-validator/crawl-and-validate	Crawl, then validate every discovered page.
GET	/api/tag-validator/crawled-urls	List of discovered URLs (JSON).
POST	/api/schedule/add	Create a scheduled audit.
GET	/api/schedule/list	List active schedules.
DELETE	/api/schedule/cancel/:id	Cancel a scheduled audit.
GET	/api/schedule/history	List past automated runs.
GET	/api/mail-config	Brevo email config status.
POST	/api/mail-config	Save Brevo API key + sender.
POST	/api/test-email	Send a test email.
GET	/api/ai/config	AI key status (Groq + Gemini).
POST	/api/ai/config	Save Groq and/or Gemini key.
POST	/api/ai/chat	Chat with the AI assistant (agentic loop).
GET	/api/ai/download/results	Assistant's Excel report download.
GET	/api/ai/download/crawled	Assistant's crawled URLs Excel.

7. Deployment

Docker image

The Dockerfile starts from `python:3.10-slim`, installs Node 20, the npm dependencies, Playwright + Chromium (and Chromium's system dependencies), copies the source, drops privileges to a non-root user, and starts `npm start` on port 7860 (the port Hugging Face Spaces expects).

Hugging Face Space

The repo is configured as a Docker-SDK Space. Pushing to the origin remote (huggingface.co/spaces/...) triggers an automatic rebuild. The chatbot's API keys must be added as Space **secrets** — GROQ_API_KEY and (optionally) GEMINI_API_KEY — because the local ai_config.json file is gitignored and not pushed.

GitHub mirror

The same repo also has a github remote for version control and code review. Pushes go to both:

```
git push github main && git push origin main
```

8. Run locally

```
npm install
pip install pandas openpyxl playwright playwright-stealth
playwright install chromium
npm start # → http://localhost:4000
```

Then in the browser:

- Click the floating message-bubble (bottom-right) to open the AI assistant.
- Tap the gear (settings) icon and paste your Groq key (and optionally a Gemini fallback key).
- Ask anything — try "is GA4 on https://www.nike.com?" or "crawl https://example.com".

9. Project layout

File	Role
server.js	Node entry point. UI API, cron, email, Python orchestration.
ai_assistant.js	Chatbot backend: tool definitions, Groq+Gemini fallback, agentic loop.
bulk_tag_validator.py	Playwright + CDP tag audit (Tealium/Adobe/GA4/GTM/pixels) + CMS/HTTP detection.
domain_crawler.py	Sitemap + BFS same-domain crawler with URL canonicalisation.
public/index.html	Single-page UI: navbar, three tabs, floating chat widget.
public/app.js	Frontend logic: tabs, polling, table rendering, chat widget.
Dockerfile	Image: Python 3.10 + Node 20 + Playwright Chromium.
package.json / requirements.txt	Node and Python dependency manifests.

Generated for the Tag Validator Pro project. See the repository for full source and history.